Assignment 3

100%

17.9.2024

Poengsum frakoblet modus:
100%

Legg til kommentar

Anonym vurdering: **nei**

13.10.2024

Detaljer

Introduction

You are to design and implement a simple guestbook page. Visitors can see the existing entries or add their own. This will require the use of Entity Framework to create and store models, and Razor scripting in the main view to show the existing entries.

Deliver on your personal git repository under the directory “assignment_3”.

Tip: Create the project with *Individual Authentication*. This project template already has a database context set up so you don't have to do it manually.

Note: Your project **MUST** have a database initializer that does .EnsureDeleted() and .EnsureCreated(). Otherwise the tests won't find a database.

Requirements

Create a new controller called *GuestbookController* and place the guestbook page with the current guestbook entries on the *Index* action. Add a link on this page to another action called *Add*. This is the page with the form to add a new entry in the guestbook.

Guestbook/Add view

- Name, title and message must be sent in a post request from an HTML form
- All fields in the form are required:
- Name field must have id: “Name"
- Title field must have id: “Title”, minimum length 5, max length 50
- Message must have id: “Message”, minimum length 20, max length 200.
  This field should support newlines, e.g. HTML textarea
- Button must have id: "Submit"

If the post is successful (valid and added to the database) the user should be redirected to the *Index* page of the guestbook. Otherwise the form should be left as it was so that the user can correct their error (e.g. missing name).

Tip: If you name the fields in your model according to the requirements above the form names will automatically be correct when you use the *asp-for* tag attribute. See the Entity Framework lecture for details.

Guestbook/Index view

The guestbook/Index page must show existing entries. Try to organize the HTML, CSS and C# in a clean and tidy way. You can display the names, titles and messages any way you like, but all must be present.

Since the input to this view is a list of entries, you can use *@model IEnumerable<DataType>* when declaring the model. See the lecture for details.

Areas of interest

For the client you will need to design the user interface as you did in assignment 2; using HTML and CSS. The HTML form will then send the form contents to the server. On the server side you need an MVC controller to receive this POST request in an MVC model and store it to the database using a database context.

Testing and delivery

See *Local testing* and *Delivery and Bamboo testing* under *Assignments*.